ESIR-SYS1 TP 5 Assembleur: Réaliser un compteur (TP Non noté)

François Taiani (`ftaiani.ouvaton.org`)*

1 Préliminaires

1.1 Préambule

- Créez tout d'abord un répertoire `tpasm` dans votre répertoire maison pour y sauvegarder les fichiers de vos TP d'assembleur. Créez ensuite un répertoire `tp5` dans le répertoire `tpasm`.
- Vérifiez que vous arrivez à faire fonctionner `nasm` et l'éditeur de lien `ld` en compilant puis en exécutant l'exemple `helloworld.asm` que nous avons vu dans l'unité 6. Le code de l'exemple est disponible sur le site Moodle su cours, ainsi que dans le répertoire partagé `/share/esir1/sys/tpasm/`.

  Copiez le fichier `helloworld.asm` dans votre répertoire `tp5` puis compilez le en utilisant les commandes `nasm` et `ld`:

  ```
  $ nasm -felf64 helloworld.asm
  $ ld helloworld.o -o helloworld
  $
  ```

  Vérifiez que `./helloworld` imprime son message et s'exécute sans erreur.
- Comme pour les TPs de Shell, vous êtes libre d'utiliser l'éditeur de votre choix pour ce TP (`nano`, `emacs`, `gedit`). Privilégiez cependant un éditeur capable de coloration syntaxique (`emacs`, `gedit`), en utilisant une police à chasse fixe (*fixe-width font*) comme `courrier`, qui rendent la programmation plus aisée.

1.2 But du TP

Le but de cet exercice est de réaliser en assembleur un compteur qui prend les valeurs `0` à `19`, et imprime ces valeurs en format décimal sur la console. Le TP comprend trois versions de ce compteur: une version de base (`simple_counter`), qui constitue le cœur du TP, et deux versions avancées (`simple_counter_with_pause` et `simple_counter_with_pause_and_bar`). Les deux versions avancées sont optionnelles, et proposées aux binômes qui termineraient rapidement la version de base.

Vous pouvez visualiser l'exécution de ces trois versions sur le site du cours.

2 Travail demandé

2.1 Partie I : Compteur Simple (programme `simple_counter.asm`)

2.1.1 Jalon 1: Compteur de `0` à `9`

Réalisez un programme qui imprime les nombres de `0` à `9`, en imprimant chaque valeur sur une nouvelle ligne. Votre programme devra utiliser une boucle, et utiliser l'appel système n. 1 (`write`) pour imprimer chaque chiffre.

Un code équivalent Java de ce programme serait:

*francois.taiani@irisa.fr

```
for(int i=0;i<=9;i++) {
   System.out.println(i);
}
```

Quelques conseils:

- Progressez par étapes. Commencez par exemple par écrire un programme qui n'imprime que `0`, une seule fois, sans boucle. Modifier ensuite ce premier programme pour imprimer `0` dix fois à la suite en utilisant une boucle. Enfin modifier encore ce programme pour imprimer les chiffres de `0` à `9`. Vous pouvez vous inspirer des exemples de code vus en cours.

  Vérifiez entre chaque étape que votre programme compile et s'exécute correctement.
- Pensez à sauvegarder puis à restaurer les registres que vous souhaitez préserver lors de l'appel système, en utilisant la pile (`push` et `pop`). Attention: les registres sont restaurés de la pile dans l'ordre inverse de celui dans lequel ils ont été sauvegardés.
- N'oubliez pas de commenter suffisamment votre code, pour garder une trace de la sémantique des registres, et des structures que vous avez utilisées (boucles).
- Pour plus de lisibilité vous pouvez encapsuler l'impression d'un chiffre dans une fonction, en passant la valeur du chiffre à imprimer dans un registre.

2.1.2 Jalon 2: Compteur de `00` à `19`

Étendez le programme précédent pour imprimer les nombres de `0` à `19`, en format décimal, sur deux chiffres (soit de `00` à `09`). Pour ne pas avoir à calculer la représentation décimale d'un nombre binaire nous allons utiliser deux boucles imbriquées, comme dans cet exemple en Java:

```
for(int j=0;j<=1;j++) {
  for(int i=0;i<=9;i++) {
    System.out.println(Integer.toString(j)+Integer.toString(i));
  } // EndFor i
} // EndFor j
```

Quelques conseils:

- Vous aurez besoin de deux registres distincts pour chacune des boucles.
- Vous devez maintenant calculer le code ASCII de 2 chiffres, et construire une chaîne de caractère comprenant trois caractères: le chiffre des dizaines, le chiffre des unités, et le caractère de saut à la ligne (`\n` en Java en C, code ASCII 10 en assembleur).

Bonus

- Si vous êtes d'humeur aventurière, vous pouvez essayer de réaliser le compteur en n'utilisant qu'une seule boucle. Vous devrez alors calculer le nombre de dizaines et d'unités de votre compteur comme dans cette version en Java:

  ```
  for(int i=0;i<=19;i++) {
    int tens  = i/10 ;
    int units = i%10 ;
    System.out.println(Integer.toString(tens)+Integer.toString(units));
  } // EndFor i
  ```

  Vous aurez besoin d'utiliser l'instruction `div`. Sur 16 bits, `div` divise la valeur de `ax` par l'opérande (un registre ou une mémoire, ça ne peut pas être une constante/valeur immédiate), et place le quotient dans `al`, et le reste dans `ah`.

2.2 Partie II (optionnelle) : Ajout de pauses (`simple_counter_with_pause`)

Le compteur a le même comportement que précédemment mais marque des pauses de 0,5 seconde entre chaque incrément. Pour un effet esthétique, le script ne saute plus à la ligne entre chaque valeur (avec `\n`, ASCII code 10), mais revient en début de ligne, sans sauter de ligne (avec `\r`, ASCII code 13, caractère de retour chariot, ou *carriage return*), avant d'imprimer chaque nouvelle valeur.

Le code équivalent en Java devient:

```
for(int j=0;j<=1;j++) {
  for(int i=0;i<=9;i++) {
    System.out.print("\r"+Integer.toString(j)+Integer.toString(i));
    Thread.sleep(500); // 0.5 second
  } // EndFor i
} // EndFor j

System.out.println(); // final newline
```

Pour réaliser la pause vous aurez besoin d'invoquer l'appel système `nanosleep`, qui porte le numéro 35 sur Linux 64 bits. `nanosleep` prend deux pointeurs comme paramètres:

- Un pointeur vers un couple de valeurs (secondes, nanosecondes), encodées en 64bits (ce qu'on appelle un `struct` en C);
- Un second pointeur utilisé en cas d'interruption, que nous mettrons à zéro (`null`) ici pour ne pas l'utiliser. (Faire `man nonsleep` pour plus d'informations.)

Ces deux pointeurs doivent être passés dans les registres `rdi` et `rsi`. `rax` doit lui contenir le numéro de l'appel système (35 ici).

Par exemple, si vous avez défini les deux entiers de 64 bits suivants dans votre section de données

```
timeval:          ; struct needed to call nanosleep system call
  dq 2            ; 2 seconds, dq means "define quadwords" = integers on 8 bytes
  dq 0            ; 0 nanoseconds
```

vous pouvez ensuite appeler `nanosleep` avec:

```
mov rax, 35       ; syscall nanosleep for x86_64, see man nanosleep
mov rdi, timeval  ; pointing to struct encoding duration of sleep
mov rsi, 0        ; null means 2nd parameter not used
syscall
```

Quelques conseils:

- Votre code va progressivement se complexifier. Pensez à faire ressortir les différentes parties de votre code en utilisant des lignes de commentaire, ne contenant aucune instruction. Par exemple:

  ```
  ;; EXITING PROGRAM
  mov       rax, 60                 ; system call for exit
  mov       rdi, 0                  ; exit code 0
  syscall                           ; invoke operating system to exit
  ```

- Vous pouvez factoriser l'appel à `nanosleep` dans une fonction pour un code plus modulaire.

2.3 Partie III :(optionnelle) : Barre de progression (`simple_counter_with_pause_and_bar.asm`)

Dans cette dernière version, l'on souhaite afficher une barre de progression faites d'astérisques (`*`) au fur et à mesure de la progression du compteur.

Le code équivalent Java devient:

```
String bar = "\r**********";
int count = 0;

for(int j=0;j<=1;j++) {
  for(int i=0;i<=9;i++) {
    count++;
    System.out.print(bar.substring(0,count+1)+" "+
                     Integer.toString(j)+Integer.toString(i));
    Thread.sleep(500); // 0.5 second
  } // EndFor i
} // EndFor j

System.out.println(); // final newline
```

Quelques conseils:

- Pour réaliser cette version, vous pouvez comme dans le code ci-dessus, pré–définir la barre complète (contenant 20 astérisques), et utiliser le fait que l'appel système `write` que nous utilisons pour imprimer sur la console prend en paramètre le nombre de caractères à imprimer (dans `rdx`). En faisant varier `rdx`, vous imprimerez des préfixes plus ou moins grands de votre barre pré–définie.
- Le fait de placer un retour chariot en début de la chaîne `bar` dans le code Java permet de revenir à la ligne à chaque incrément avant d'imprimer la bar puis la nouvelle valeur du compteur.
- Si vous utiliser deux boucles imbriquées (comme dans l'exemple Java) vous aurez besoin d'un compteur général (`count` dans le code ci-dessus) pour compter le nombre total d'itérations effectuées. (Utilisez un registre pour ce compteur général.) Si vous n'avez qu'une seule boucle (voir le bonus de la partie I), vous pouvez directement utiliser l'index de cette boucle globale.
- Dans tous les cas, pensez à bien sauvegarder et restaurer les registres nécessaires avant les appels système, ou avant les appels aux fonctions que vous aurez définies.

— Fin du sujet de TP —